TRISHA

Product Features Documentation

Document	Product Features Specification
Project	TRISHA — Text Recognition and Intelligent Speech Helper Assistant
Layer coverage	Flutter Mobile App · Express.js Backend API · React Web Portal
Priority system	P1 Must Have · P2 Should Have · P3 Nice to Have (MoSCoW)
Institution	National University Philippines — BSIT Capstone
Total features	8 Mobile · 8 Backend · 8 Web = 24 total

Table of Contents

1. File & Document Manifest

All files to be created as part of this export

20 documents across PDF, DOCX, Markdown, JSON, CSV, and PPTX formats

The following table lists every file and document that should exist in the TRISHA project repository and submission package. Files are organized by type: infrastructure documents (PDFs, this spec), source code documentation (READMEs, .env templates), API tooling (Postman collection), design artifacts (ERD, wireframes), Agile evidence (backlog export, burndowns), test results, the five thesis chapters, and the defense presentation.

#	File / Document	Format	Purpose
1	`TRISHA_Product_Features.pdf`	PDF	This document — full product features specification for all layers (mobile, backend, web
2	`TRISHA_System_Architecture.pdf`	PDF	System architecture, deployment diagram, data flow, security, and scalability (already g
3	`README.md (root repo)`	Markdown	Repo overview, setup instructions, folder structure, and links to each sub-project
4	`README.md (/flutter)`	Markdown	Flutter app: install, run, build APK, environment setup, dependencies list
5	`README.md (/backend)`	Markdown	Express API: install, .env variables, run dev, run tests, Postman collection link
6	`README.md (/web)`	Markdown	React web: install, .env setup (VITE_API_URL), run dev, deploy to Vercel
7	`.env.example (/backend)`	Text	Template for all backend environment variables (MongoDB URI, JWT secret, Cloudinar
8	`.env.example (/web)`	Text	Template for React env vars: VITE_API_URL pointing to Express backend
9	`TRISHA_API_Collection.json`	JSON	Postman collection: all 20+ API endpoints with sample headers, bodies, and environme
10	`TRISHA_ERD.png / .pdf`	PNG/PDF	Entity-Relationship Diagram for all 6 MongoDB collections with field types and relations
11	`TRISHA_Wireframes.pdf`	PDF	Flutter UI wireframes: home/scan screen, history, AI chat, settings — annotated with ac
12	`TRISHA_Agile_Board_Export.csv`	CSV	Sprint backlog export: all stories with ID, epic, priority, estimate, status, assignee
13	`TRISHA_Sprint_Burndowns.pdf`	PDF	Sprint burndown chart screenshots per sprint — evidence for Chapter 3 (Methodology)
14	`TRISHA_Test_Results.pdf`	PDF	OCR accuracy test results, API response time benchmarks, TTS latency, Lighthouse sc
15	`TRISHA_Chapter1.docx`	DOCX	Introduction, background of the study, statement of the problem, objectives, scope and
16	`TRISHA_Chapter2.docx`	DOCX	Review of related literature and studies (local and foreign) — 15–20 sources, with synth
17	`TRISHA_Chapter3.docx`	DOCX	Methodology: Agile SDLC, system design, data flow, tools used, testing approach
18	`TRISHA_Chapter4.docx`	DOCX	Results and discussion: screenshots, OCR accuracy table, user testing feedback, AI re
19	`TRISHA_Chapter5.docx`	DOCX	Conclusions, recommendations, limitations, future work (e.g., live OCR, sign language
20	`TRISHA_DefenseSlides.pptx`	PPTX	Panel defense presentation: problem, solution, demo flow, architecture, results, Q&A pr

Files marked DOCX are thesis chapter documents for NU submission. The PDF documents (this file and the architecture doc) serve as technical annexes. The Postman collection, ERD, and wireframes are required evidence for Chapter 3 (Methodology).

2. Flutter Mobile App Features

Flutter Mobile App — 8 Features

Core accessibility app · On-device OCR · Filipino TTS · Voice-first UI

The Flutter APK is the primary deliverable of TRISHA. It is designed to be operated entirely without visual interaction — all functions are accessible via voice commands or large-target gestures. Features are listed below in implementation priority order (P1 → P2).

MOB-01	Point-and-Read OCR	P1 — Must Have
Description	The core interaction loop of the app. The user opens the camera (or picks from gallery), captures an image of printed text, and the app immediately extracts the text using Google ML Kit and reads it aloud via TTS — with zero visual interaction required.	
Acceptance criteria	• Camera opens on app launch with a single tap or voice command 'scan'. • ML Kit extracts text within 2 seconds for standard A4 printed document at 12MP. • Extracted text is read aloud automatically via flutter_tts within 300ms of extraction. • If text extraction returns empty, TTS reads 'Walang nahanap na teksto. Subukan ulit.' • Gallery picker works as an alternative to live camera capture.	
Tech / tools	`google_ml_kit ^0.16 · image_picker ^1.0 · flutter_tts ^3.8`	
Notes	*ML Kit runs fully on-device — no internet required. This is the primary offline feature.*	

MOB-02	Filipino / English TTS	P1 — Must Have
Description	Text-to-speech output supporting both Filipino (Tagalog) and English languages. Language is auto-detected from scan content or manually set in user settings. This is TRISHA's primary novelty differentiator — no existing accessibility app targets Filipino TTS for visually impaired users.	
Acceptance criteria	• flutter_tts correctly pronounces Filipino words using the 'fil-PH' language pack. • User can switch language between Filipino and English from settings or via voice command 'English mode' / 'Filipino mode'. • TTS speed is adjustable from 0.5x to 2.0x, defaulting to 1.0x. • TTS voice pitch is adjustable for user preference. • TTS works offline using the device's built-in TTS engine (Google TTS on Android).	
Tech / tools	`flutter_tts ^3.8 · Language: fil-PH (Filipino), en-US (English) · Tested on Google TTS engine`	
Notes	*iOS Siri TTS may not support Filipino — test on Android primary. Document this limitation in Chapter 5.*	

MOB-03	Voice Command Navigation	P1 — Must Have
Description	Full app control via spoken commands. The user never needs to look at or touch the screen. All primary actions — scanning, reading, explaining, saving, navigating history — are accessible via voice. Enables true eyes-free accessibility.	
Acceptance criteria	• App listens for commands continuously when in active mode (indicated by a haptic buzz). • Commands supported: 'scan' / 'kumuha', 'basa' / 'read', 'stop', 'ipaliwanag' / 'explain', 'i-save' / 'save', 'history', 'ulit' / 'repeat', 'settings'. • Unrecognized commands trigger a TTS prompt: 'Hindi ko naintindihan. Sabihin ulit.' • Response latency from command to action is under 1 second on mid-range Android devices.	
Tech / tools	`speech_to_text ^6.3 · Language models: fil-PH, en-US · Requires RECORD_AUDIO permission`	

MOB-04	AI Explain Mode	P1 — Must Have
Description	After a scan is read aloud, the user can say 'Ipaliwanag' or tap the Explain button to ask the AI to explain the content in simple terms. The AI response is read aloud via TTS. Users can ask follow-up questions conversationally. Powered by OpenAI GPT-4o-mini via the Express backend proxy.	
Acceptance criteria	• AI explain is triggered by voice command 'ipaliwanag' / 'explain' or button tap after any scan. • API call goes to POST /api/ai/explain — API key never exposed in Flutter source. • AI response is streamed or returned in full and read aloud by TTS automatically. • Conversation context is maintained for follow-up questions within the same scan session. • AI conversation is saved to ai_conversations collection in MongoDB for history review. • If offline or API fails, TTS reads: 'Hindi available ang AI ngayon. Subukan kapag may internet.'	
Tech / tools	`POST /api/ai/explain (Express proxy) · OpenAI GPT-4o-mini · http ^1.2 package`	
Notes	*AI features require internet. Never call OpenAI directly from Flutter — always proxy through Express to protect the API key.*	

MOB-05	Offline Mode	P1 — Must Have
Description	Core OCR and TTS functions work fully without internet. When offline, scans are queued locally and synced to the backend when connectivity is restored. The offline state is communicated to the user via TTS.	
Acceptance criteria	• MOB-01 (OCR) and MOB-02 (TTS) complete successfully with airplane mode enabled. • Offline scans are stored locally in Hive or SharedPreferences with isPendingSync: true. • A sync indicator is announced via TTS when connectivity is restored: 'Na-sync na ang iyong mga scan.' • History of offline scans is accessible even without internet.	
Tech / tools	`hive ^2.2 · connectivity_plus ^5.0 · ML Kit (on-device) · flutter_tts (device TTS engine)`	

MOB-06	Scan History	P2 — Should Have
Description	All completed scans are saved to the backend and accessible via voice command or gesture. Users can navigate history, re-read any past scan, and play back the AI explanation for that scan session.	
Acceptance criteria	• Voice command 'history' opens the scan history screen, read aloud as a numbered list. • User can say 'Item 3' or swipe to select a history item. • Selecting a history item reads the extracted text aloud. • History is paginated — fetches 20 items at a time from GET /api/scan/history. • Delete a scan via voice command 'burahin' or long-press gesture.	
Tech / tools	`GET /api/scan/history · DELETE /api/scan/:id · ListView with Semantics widgets for accessibility`	

MOB-07	Gesture Navigation UI	P2 — Should Have
Description	The app UI is designed for zero-vision use: oversized tap targets (minimum 48x48dp per WCAG), swipe navigation, long-press for secondary actions, and haptic feedback on all interactions. No small buttons, no fine motor control required.	
Acceptance criteria	• All interactive elements meet 48x48dp minimum touch target size (WCAG 2.5.5). • Swipe left = previous scan, swipe right = next scan in history. • Double tap = repeat current TTS output. • Long press on scan card = show context menu (read, explain, save, delete) — read aloud. • Haptic feedback on: app launch, scan capture, TTS start, command recognized, error. • All UI elements have Semantics labels for Android TalkBack compatibility.	
Tech / tools	`GestureDetector · Semantics widget · HapticFeedback · flutter_accessibility_service (optional)`	

MOB-08	User Settings	P2 — Should Have
Description	Persistent user preferences for TTS speed, pitch, language, and font size (for sighted helpers/caregivers viewing the screen). Settings sync to the backend user profile.	
Acceptance criteria	• Settings screen accessible via voice command 'settings' or bottom nav. • TTS speed slider: 0.5x to 2.0x, default 1.0x — change is applied immediately. • Language toggle: Filipino / English. • Font size setting affects all on-screen text (for sighted helper viewing). • Settings are saved to PATCH /api/user/settings and restored on login.	
Tech / tools	`PATCH /api/user/settings · SharedPreferences for local cache · flutter_tts setRate/setPitch`	

3. Express.js Backend API Features

Express.js Backend — 8 Features

Node.js 20 LTS · MongoDB Atlas · JWT Auth · Cloud OCR · AI Proxy

The Express.js API is the data and intelligence layer of TRISHA. It handles authentication, image storage, OCR cloud fallback, AI proxying, and all admin operations. All endpoints are RESTful, JWT-protected where appropriate, and documented in the Postman collection (FILE-09 in the manifest above).

API-01	JWT Authentication System	P1 — Must Have
Description	Complete user authentication: registration, login, and token-based session management. Passwords are hashed with bcrypt. JWTs are short-lived (1 hour) with optional refresh token support. Role-based middleware protects all non-public routes.	
Acceptance criteria	• POST /api/auth/register creates user, hashes password with bcrypt (rounds=12), returns JWT. • POST /api/auth/login validates credentials, returns JWT + refresh token. • All protected routes return 401 if no token, 403 if token valid but role insufficient. • JWT secret is a 256-bit random string stored in .env — never hardcoded. • Passwords are never returned in any API response.	
Tech / tools	`jsonwebtoken ^9 · bcrypt ^5 · express-validator · Custom verifyToken() and requireRole() middleware`	

API-02	OCR Cloud Fallback	P1 — Must Have
Description	When ML Kit confidence is insufficient (complex layouts, low-quality scans, handwriting), the Flutter client sends the image to POST /api/scan/ocr-cloud which processes it via Google Vision API and returns enhanced results.	
Acceptance criteria	• POST /api/scan/ocr-cloud accepts multipart/form-data image upload. • Google Vision TEXT_DETECTION is called and structured response is returned. • If Vision API fails or quota exceeded, returns a structured error with fallback message. • GCP project has a $0 hard budget cap to prevent unexpected charges. • Usage is logged per user in scan_history.ocrEngine field ('mlkit' or 'vision').	
Tech / tools	`@google-cloud/vision ^4 · multer ^1.4 · GCP project with billing account (for free tier access)`	
Notes	*Google Vision requires a billing account even for free tier. Set hard $0 budget cap in GCP Console.*	

API-03	AI Proxy Endpoint	P1 — Must Have
Description	Secure server-side proxy to OpenAI GPT-4o-mini (or Gemini 1.5 Flash). The Flutter client never holds the AI API key. Conversation context is passed per request and the full exchange is persisted to MongoDB.	
Acceptance criteria	• POST /api/ai/explain accepts { text, question, conversationId } body. • System prompt instructs model to respond in the same language as the user's question (Filipino or English). • Full AI response is returned to client and saved to ai_conversations collection. • If AI API is unavailable, returns 503 with a user-friendly message. • Endpoint is rate-limited to 20 requests per user per hour to control API costs.	
Tech / tools	`openai ^4 (or @google/generative-ai) · Rate limit: express-rate-limit per userId · Model: gpt-4o-mini`	

API-04	Scan Management API	P1 — Must Have
Description	Full CRUD for user scan history. Images are uploaded to Cloudinary; metadata and extracted text are saved to MongoDB. Supports pagination, filtering, and deletion.	
Acceptance criteria	• POST /api/scan/upload accepts image + extractedText + ocrEngine, uploads to Cloudinary, saves to scan_history. • GET /api/scan/history returns paginated list (20/page) ordered by createdAt DESC. • GET /api/scan/:id returns single scan with imageUrl and extractedText. • DELETE /api/scan/:id deletes MongoDB record and Cloudinary asset (using public_id). • All endpoints require valid JWT and return only the authenticated user's scans.	
Tech / tools	`cloudinary ^2 · multer-storage-cloudinary · Mongoose scan_history model · Query: { userId, page, limit }`	

API-05	User Profile & Settings API	P1 — Must Have
Description	Endpoints for reading and updating user profile and TTS preferences. Settings are stored in the users.settings subdocument in MongoDB and synced to the Flutter app on login.	
Acceptance criteria	• GET /api/user/profile returns { name, email, role, organizationId, settings }. • PATCH /api/user/settings accepts { ttsSpeed, language, fontSize } and updates MongoDB. • Settings are returned in the login response so the app can apply them immediately. • Password change requires current password confirmation before update.	
Tech / tools	`Mongoose users model · settings subdocument: { ttsSpeed: Number, language: String, fontSize: Number }`	

API-06	Superadmin Management API	P1 — Must Have
Description	Admin-only endpoints for managing all users, organizations, and system data across all tenants. Protected by requireRole('superadmin') middleware on every route.	
Acceptance criteria	• GET /api/admin/users returns paginated, searchable, filterable list of all users. • PATCH /api/admin/user/:id can update role, isActive status, and organizationId. • GET /api/admin/organizations returns all orgs with member count and admin info. • POST /api/admin/organizations creates a new organization tenant. • GET /api/admin/logs returns audit log entries (login, role change, deletion) newest-first. • All admin actions are written to admin_logs collection with actor userId and timestamp.	
Tech / tools	`requireRole('superadmin') middleware · Mongoose aggregate for counts · admin_logs schema`	

API-07	Feedback API	P2 — Should Have
Description	Users can submit quality reports for scan results. Feedback is stored and reviewable by admins via the React portal. Used for tracking OCR accuracy issues and TTS problems.	
Acceptance criteria	• POST /api/feedback accepts { scanId, rating, issue, notes } — requires JWT. • GET /api/feedback (admin only) returns paginated feedback filtered by status. • PATCH /api/feedback/:id allows admin to update status to 'reviewed' or 'resolved'. • issue field: enum of ['wrong_text', 'poor_tts', 'app_crash', 'other'].	
Tech / tools	`Mongoose feedback model · Status enum: open | reviewed | resolved`	

API-08	Rate Limiting & Security Middleware	P2 — Should Have
Description	Production-grade Express middleware stack: rate limiting, CORS, Helmet headers, and request validation. Protects the API from abuse and common web vulnerabilities.	
Acceptance criteria	• express-rate-limit: 100 requests per 15 minutes per IP on all routes. • Stricter limit on auth routes: 10 requests per 15 minutes per IP. • CORS allows only: Flutter app origin, Vercel React URL, and localhost for development. • Helmet.js sets all recommended security headers (Content-Security-Policy, X-Frame-Options, etc.). • All request bodies validated with Joi or Zod before reaching controller.	
Tech / tools	`express-rate-limit ^7 · cors ^2 · helmet ^7 · joi ^17 (or zod ^3)`	

4. React Web Portal Features

React Web Portal — 8 Features

Vite 5 · TailwindCSS 3 · Public info site (GCash-style) · Superadmin dashboard

The React web portal serves two distinct audiences: the general public (via the GCash-style information site) and system administrators (via the protected superadmin dashboard). Both are deployed to Vercel from the same repository, separated by route: public routes at / and admin routes at /admin/*.

WEB-01	Public Information Site (GCash-style)	P1 — Must Have
Description	A polished public-facing marketing website for TRISHA — modeled after GCash.com in structure and visual quality. It serves as the project's digital home: communicates the app's value to stakeholders, provides the APK download, and establishes credibility for the panel presentation.	
Acceptance criteria	• Hero section: app name, tagline ('Marinig ang Mundo'), and a prominent Download APK CTA button. • Features grid: 6 icons with short labels — Point & Read, AI Explain, Filipino TTS, Voice Control, Offline Mode, Scan History. • How It Works: 3-step illustrated flow (Capture → Extract → Listen). • About section: project background, NU branding, team member names, client organization logo. • FAQ accordion: 5–6 questions and answers about the app and its accessibility features. • Footer: team names, course, academic year, NU logo, GitHub link. • Site achieves Lighthouse performance score >85 and accessibility score >90.	
Tech / tools	`React 18 · Vite 5 · TailwindCSS 3 · react-scroll (smooth anchor nav) · Deployed on Vercel`	

WEB-02	Superadmin Authentication	P1 — Must Have
Description	Protected login for the admin portal. JWT token is stored in memory (not localStorage) and attached to all API requests. Automatic redirect to login if token expires. Only users with role 'superadmin' can access the portal.	
Acceptance criteria	• Login page at /admin/login — email + password form, calls POST /api/auth/login. • JWT stored in React state (or httpOnly cookie) — never in localStorage. • All /admin/* routes are guarded by a ProtectedRoute component that checks role === 'superadmin'. • On token expiry (401 response), user is redirected to login with a toast: 'Session expired.' • Logout clears token and redirects to /admin/login.	
Tech / tools	`React Router v6 · Axios interceptors for 401 handling · Context API or Zustand for auth state`	

WEB-03	Admin Dashboard	P1 — Must Have
Description	Overview page with metric cards and charts showing platform health. The first screen the superadmin sees after login.	
Acceptance criteria	• Metric cards: Total Users, Scans Today, AI Queries Today, Active Organizations. • Line chart: scans per day for the last 30 days. • Bar chart: top 5 most active users by scan count. • Recent feedback table: last 5 open items with status badges. • All data fetched from admin API endpoints on mount with a loading skeleton state.	
Tech / tools	`recharts ^2 or chart.js · GET /api/admin/stats · Skeleton loaders for loading state`	

WEB-04	User Management	P1 — Must Have
Description	Full CRUD interface for managing all registered users across all tenants. Supports search, filter by role/organization, role assignment, and deactivation.	
Acceptance criteria	• Searchable, filterable data table with columns: Name, Email, Role, Organization, Status, Joined. • Role dropdown per row: user | org_admin | superadmin — triggers PATCH /api/admin/user/:id. • Toggle user active/inactive — inactive users cannot log in. • Pagination: 25 users per page with prev/next controls. • Export to CSV button (client-side, from current filtered data).	
Tech / tools	`GET /api/admin/users · PATCH /api/admin/user/:id · TanStack Table or plain HTML table`	

WEB-05	Organization / Tenancy Management	P2 — Should Have
Description	Create and manage organizations (tenants). Each organization represents an institutional client (school, NGO). Org admins manage users within their org. Superadmin has visibility across all.	
Acceptance criteria	• Organization list with: Name, Type, Admin, Member Count, Created Date. • Create org modal: name, type (school | ngo | government | other), assign org_admin. • Click org row to drill down: view all members, their roles, and scan activity. • Deactivate organization — all members lose access until reactivated.	
Tech / tools	`GET/POST /api/admin/organizations · Modal with react-hook-form · Nested route for org detail`	

WEB-06	Scan Logs Viewer	P2 — Should Have
Description	Admin view of all scan records across all users. Supports filtering by user, organization, date range, and OCR engine. Clicking a scan shows the image preview and extracted text.	
Acceptance criteria	• Table columns: User, Organization, OCR Engine, Text Preview (50 chars), Date. • Filter bar: date range picker, user search, OCR engine select (mlkit | vision). • Click row → modal showing full Cloudinary image + full extracted text. • Pagination: 25 per page. Total count shown.	
Tech / tools	`GET /api/admin/scans (superadmin endpoint) · Cloudinary image URL in modal · date-fns for formatting`	

WEB-07	Feedback Management	P2 — Should Have
Description	Admin interface for reviewing user-submitted OCR quality reports. Filter by status, mark as reviewed or resolved, and add admin notes.	
Acceptance criteria	• Table: User, Issue Type, Rating (stars), Status badge, Submitted Date. • Filter by status: All | Open | Reviewed | Resolved. • Click row → detail panel with full feedback notes and linked scan preview. • PATCH /api/feedback/:id to update status. Confirmation toast on save.	
Tech / tools	`GET/PATCH /api/feedback · Status badge colors: open=amber, reviewed=blue, resolved=green`	

WEB-08	Audit Log Viewer	P3 — Nice to Have
Description	Read-only chronological log of all system-level actions: user logins, role changes, deletions, and org modifications. Provides accountability and a paper trail for institutional clients.	
Acceptance criteria	• Table columns: Actor (user), Action, Target, Timestamp. • Filter by action type: login | role_change | delete | org_change. • Read-only — no edit or delete actions available on the log. • Export to CSV for institutional audit purposes.	
Tech / tools	`GET /api/admin/logs · admin_logs MongoDB collection · superadmin-only route guard`	

This document covers all 24 product features across the three layers of the TRISHA system. Each feature includes its acceptance criteria, the specific technologies and packages required, and any implementation notes critical for the capstone phase. P1 features must be fully functional for the panel defense. P2 features should be complete but may have minor gaps. P3 features are bonus scope if time permits.

TRISHA Product Features Documentation v1.0 — National University Philippines BSIT Capstone. For questions on implementation, refer to the accompanying System Architecture PDF.